Fundamental 1팀 스터디 / 딥 러닝 파이토치 교과서

08
합성곱 신경망

이미지는 가까운 픽셀끼리 관련이 깊습니다. 그 주변 정보를 그대로 살려 처리하는 신경망, CNN의 이론을 세우고 손글씨 숫자 하나가 층을 지나며 어떻게 바뀌는지 따라가 봅니다.

오늘 다루는 것

1. 합성곱과 풀링: 공간적 구조, 채널, 합성곱 연산, 패딩·스트라이드, 다수 채널의 합성곱, 풀링
2. CNN으로 MNIST 분류하기: 층마다 데이터가 어떻게 바뀌는지 따라가기
3. 깊은 CNN으로 MNIST 분류하기

끝나고 나면 할 수 있는 것

1. 이미지에 다층 퍼셉트론 대신 CNN을 쓰는 이유를 "공간적 구조"로 설명할 수 있다.
2. 채널, 커널, 특성 맵, 패딩, 스트라이드, 풀링을 정의하고 출력 크기를 계산할 수 있다.
3. 입력 (1, 28, 28)이 CNN의 각 층을 지나 숫자 10개의 확률이 되기까지 모양 변화를 설명할 수 있다.

2026년 10월 7일 수요일 (10/7 자료 3/3)
발표자 설명, 코랩 실습, 퀴즈 돌아가며 풀기 순서로 진행합니다. 막힌 질문은 디스코드에 남겨 주세요.

08

합성곱 신경망

합성곱 신경망(CNN, Convolutional Neural Network)은 **이미지 처리에 탁월한 성능**을 보이는 신경망입니다. 이미지의 핵심 성질은 가까운 픽셀끼리 서로 관련이 깊다는 것입니다. 선 하나, 모서리 하나는 이웃한 몇 개의 픽셀이 함께 만들어 냅니다. CNN은 작은 창(커널)으로 이미지를 훑으며 주변 픽셀을 한꺼번에 계산하는 **합성곱 연산**으로 이 주변 정보를 반영합니다.

왜 배우나: 오늘의 학습 경로



이미지는 주변 픽셀과 관련이 깊다
공간적 구조

펴서 넣으면 그 관계가 사라진다
다층 퍼셉트론의 한계

주변을 함께 계산한다
합성곱 연산

중요한 것만 남긴다
풀링

층을 쌓아 분류한다
CNN

08-01에서 개념을 하나씩 정의하고, 08-02에서 손글씨 숫자 7 하나가 학습된 CNN의 각 층을 지나 "7일 확률 1.00"이 되기까지 따라갑니다. 08-03에서는 층을 더 깊게 쌓아 봅니다.

08-01 합성곱과 풀링

핵심 한 줄

이미지는 가까운 픽셀끼리 관련이 깊으므로, CNN은 작은 커널로 주변 픽셀을 함께 계산하는 합성곱으로 특징을 뽑고, 풀링으로 그 결과를 요약한다.

왜 배우나

06장의 다층 퍼셉트론은 그림을 한 줄로 펴서 넣었습니다. 그래도 MNIST를 97% 맞혔지만, 그 방식은 이미지의 가장 중요한 정보인 "어떤 픽셀이 어떤 픽셀 옆에 있는가"를 버립니다. 이 소챕터의 개념들(채널, 합성곱, 패딩, 풀링)은 그 정보를 살려서 처리하는 도구이고, 이미지뿐 아니라 13장에서 문장 분류(1D CNN)에도 그대로 쓰입니다.

1) 이미지의 공간적 구조와 다층 퍼셉트론의 한계

정의

공간적 구조(spatial structure)란 이미지에서 **거리가 가까운 픽셀끼리는 값이 밀접하게 연관되어 있고, 멀리 떨어진 픽셀끼리는 연관이 약하다**는 성질입니다. 선, 모서리, 곡선 같은 무늬는 모두 이웃한 몇 개의 픽셀이 함께 만들어 냅니다.

- **다층 퍼셉트론의 한계 1 (정보 유실)**: 입력이 1차원 벡터여야 하므로 이미지를 한 줄로 폅니다. 그러면 바로 위아래 이웃 픽셀이 멀리 떨어진 칸이 되어 공간적 구조가 사라집니다.
- **다층 퍼셉트론의 한계 2 (변형에 약함)**: 같은 글자라도 조금 휘거나 옆으로 이동하면 픽셀 값이 완전히 달라져, 사실상 다른 입력이 됩니다.
- **합성곱 신경망(CNN)**은 이미지를 펴지 않고 2차원 그대로 받아, 작은 영역(주변 픽셀)을 함께 계산합니다. 그래서 공간적 구조를 보존하고, 같은 무늬가 어디에 있든 찾아낼 수 있어 **이미지 처리에 탁월**합니다.
- CNN은 크게 **합성곱층**(합성곱 + 활성화 함수)과 **풀링층**으로 특징을 뽑고, 마지막에 **전결합층**으로 분류합니다.

예시 적용

MNIST 손글씨 7에서 픽셀 하나를 골라 봅니다. 2차원 그림에서는 위·아래·왼쪽·오른쪽 이웃이 바로 붙어 있지만, 784칸으로 펴면 **위 이웃은 28칸 앞, 아래 이웃은 28칸 뒤**로 흩어집니다. 다층 퍼셉트론은 이 둘이 이웃이었다는 사실을 알 수 없습니다.

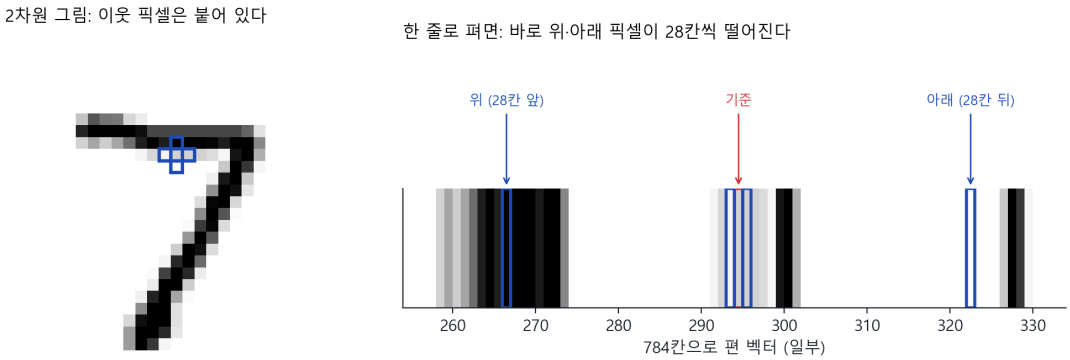


그림 1 같은 픽셀과 그 이웃(파란 칸)을 2차원 그림과 1차원으로 편 벡터에서 비교. 펴는 순간 위아래 이웃 관계가 사라진다

2) 채널

정의

이미지는 (높이, 너비, 채널) 크기의 3차원 텐서입니다. 높이는 세로 픽셀 수, 너비는 가로 픽셀 수, 채널(channel)은 한 픽셀을 표현하는 값의 개수(색 성분)입니다. 채널을 **깊이(depth)**라고도 합니다.

- **흑백 이미지**: 채널 1. 각 픽셀은 밝기 하나(0~255)
- **컬러 이미지**: 채널 3. 빨강(R), 초록(G), 파랑(B) 세 값의 조합으로 한 픽셀의 색을 만듭니다.
- 파이토치는 채널을 맨 앞에 두고, 여러 장을 묶는 배치 차원까지 붙여 **(배치, 채널, 높이, 너비)** 순서의 4차원 텐서로 받습니다.
- 합성곱층을 지나고 나면 채널은 색이 아니라 **"찾아낸 특징의 종류"**를 뜻하게 됩니다 (아래 5) 참고).

예시 적용

MNIST 손글씨는 흑백이라 (28, 28, 1), 파이토치에서는 (1, 1, 28, 28)입니다. 잔디 위의 빨간 사과 사진 같은 컬러 이미지는 (28, 28, 3)이고, 아래처럼 빨강·초록·파랑 세 장의 흑백 표로 나눠 볼 수 있습니다. 사과 부분은 빨강 채널에서 진하고, 잔디는 초록 채널에서 진합니다.

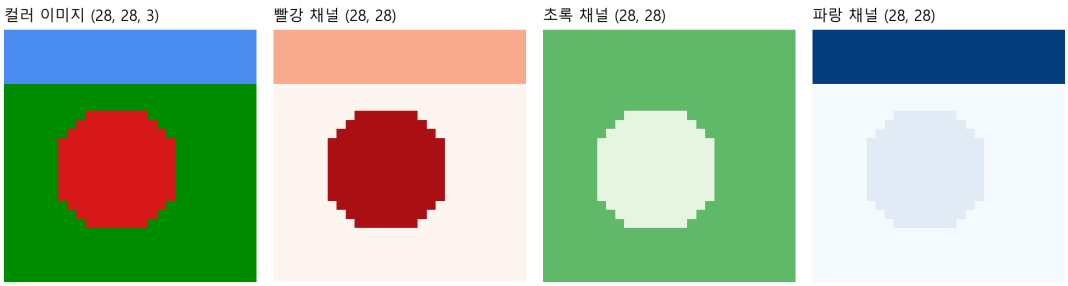


그림 2 컬러 이미지 한 장 = 빨강·초록·파랑 채널 3장. 각 채널은 그 색이 얼마나 강한지를 담은 (28, 28) 표다

3) 합성곱 연산과 가중치 공유

정의

합성곱 연산(convolution)은 작은 행렬인 **커널(kernel, 필터)**을 이미지의 왼쪽 위부터 오른쪽 아래까지 한 칸씩 밀면서, 겹치는 부분의 같은 자리끼리 곱해 모두 더하는 연산입니다. 그 결과로 만들어진 표를 **특성 맵(feature map)**이라고 합니다.

$$\text{특성 맵}(i,j) = \sum_m \sum_n \text{입력}(i+m,\ j+n) \times \text{커널}(m,n) + b$$

말로 하면: (i, j) 위치를 중심으로 한 작은 영역(주변 픽셀)과 커널을 같은 자리끼리 곱해 더한다. 그래서 출력의 한 칸에는 **그 주변의 정보가 함께** 담긴다.

- **CNN의 가중치는 커널의 원소**입니다. 사람이 정하지 않고 학습으로 정해집니다. 편향 b는 커널 하나당 하나이며 특성 맵의 모든 칸에 더해집니다.
- **지역 연결**: 출력 한 칸은 이미지 전체가 아니라 커널 크기만큼의 주변 픽셀만 봅니다. 이것이 공간적 구조를 반영하는 방법입니다.
- **가중치 공유**: 같은 커널을 이미지 전체에 돌려씁니다. 그래서 같은 무늬를 어디에서든 찾아내고, 학습할 가중치 수가 다층 퍼셉트론보다 훨씬 적습니다.
- 합성곱 결과에는 다층 퍼셉트론처럼 **활성화 함수(주로 ReLU)**를 적용합니다. 합성곱 + 활성화 함수를 합쳐 **합성곱층**이라고 합니다.

예시 적용

가운데에 세로줄이 있는 5×5 그림에 "세로줄을 찾는" 3×3 커널을 적용하면, 세로줄이 있는 가운데 열에서만 큰 값(6)이 나옵니다. 커널이 주변 3×3 픽셀을 함께 보기 때문입니다.

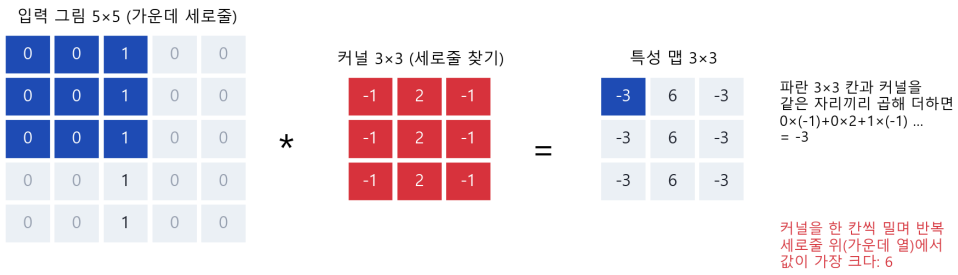


그림 3 세로줄 커널의 합성곱. 파란 3×3 영역과 커널을 곱해 더한 값이 특성 맵의 한 칸이 된다

실제 손글씨 7에 세로선·가로선 커널을 적용하면 같은 그림에서 서로 다른 무늬가 드러납니다. 가중치 공유의 효과는 숫자로도 확인됩니다. 28×28 흑백 그림에서 32개의 특징을 뽑을 때, 다층 퍼셉트론은 **25,120개**, 3×3 커널 32개짜리 합성곱층은 **320개**의 가중치만 필요합니다 (실습 2).



그림 4 실제 MNIST 숫자 3에 세로선·가로선 커널을 적용한 특성 맵과, 그 결과를 풀링한 모습

4) 패딩, 스트라이드, 특성 맵의 크기

정의

- 스트라이드(stride): 커널을 한 번에 밀어 움직이는 칸 수
- 패딩(padding): 합성곱 전에 입력 테두리에 0을 둘러 크기를 키우는 것. 출력 크기를 입력과 같게 유지할 때 씁니다.

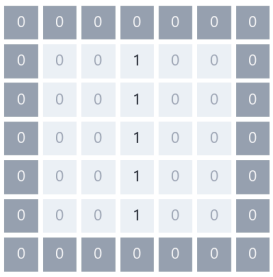
입력 크기 I, 커널 크기 K, 패딩 P, 스트라이드 S일 때 특성 맵의 크기 O는 다음과 같습니다(소수점 이하는 버림).

$$O = \left\lfloor \frac{I - K + 2P}{S} \right\rfloor + 1$$

예시 적용

- 5×5 그림, 3×3 커널, 패딩 0, 스트라이드 1이면 $(5-3+0) \div 1 + 1 = 3$, 즉 3×3
- MNIST 28×28, 3×3 커널, 패딩 1, 스트라이드 1이면 $(28-3+2) \div 1 + 1 = 28$, 크기 그대로. 08-02의 CNN이 이 설정입니다.

패딩 1: 테두리에 0을 둘러 크기를 지킨다



맥스 풀링 2×2: 칸마다 가장 큰 값만 남긴다

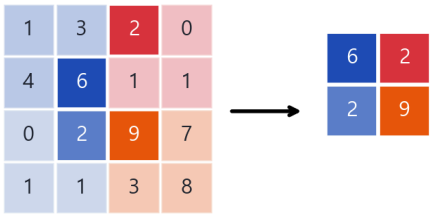


그림 5 왼쪽: 패딩 1을 두르면 5×5가 7×7이 되어, 3×3 커널 결과가 다시 5×5가 된다. 오른쪽: 맥스 풀링 (아래 6))

5) 다수의 채널을 가진 합성곱 (3차원 텐서의 합성곱)

정의

실제 입력은 채널이 여러 개인 경우가 대부분입니다(컬러 이미지, 또는 앞 층이 만든 여러 장의 특성 맵).

- 커널도 **입력과 같은 채널 수**를 가집니다. 입력 채널이 C_i개면 커널 하나의 크기는 (C_i, K, K)입니다.
- 채널마다 합성곱을 한 뒤 그 결과를 **모두 더해 특성 맵 1장**을 만듭니다(편향도 하나).
- 커널을 C_o개 쓰면 특성 맵이 C_o장 나옵니다. 즉 **출력 채널 수 = 커널 개수**입니다.
- 따라서 합성곱층의 가중치 크기는 (C_o, C_i, K, K)이고, 편향은 C_o개입니다.
- 이때 출력 채널은 색(RGB)이 아니라 **커널마다 찾아낸 서로 다른 특징**을 뜻합니다.

예시 적용

컬러 이미지 (3, 5, 5)에 3×3 커널 1개를 적용하면 커널도 (3, 3, 3)이고 결과는 (1, 3, 3) 한 장입니다. 커널을 4개 쓰면 (4, 3, 3), 4장이 됩니다. 08-02의 첫 합성곱층 `nn.Conv2d(1, 32, 3)` 은 흑백 입력(채널 1)에 커널 32개를 써서 특성 맵 32장을 만들고, 두 번째 층 `nn.Conv2d(32, 64, 3)` 은 그 32장을 입력으로 받아 커널 64개로 64장을 만듭니다.

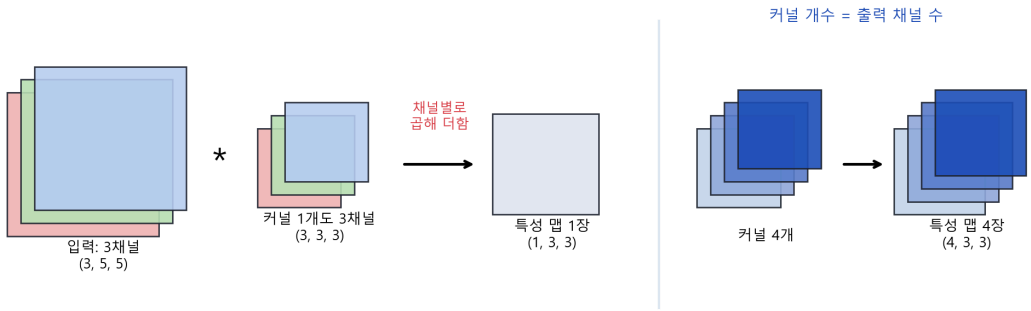


그림 6 3채널 입력과 3채널 커널의 합성곱은 채널별로 곱해 더해 특성 맵 1장이 된다. 커널이 4개면 특성 맵도 4장

6) 풀링

정의

풀링(pooling)은 특성 맵을 작은 구역으로 나눠 구역마다 값 하나만 남겨 크기를 줄이는 연산입니다. 보통 합성곱층 다음에 둡니다.

- **맥스 풀링**: 구역에서 가장 큰 값, **평균 풀링**: 구역의 평균을 남깁니다. 2×2 구역, 스트라이드 2면 높이·너비가 절반이 됩니다.
- 학습할 가중치가 없고, 채널 수는 바뀌지 않습니다.
- "이 근처에 이 특징이 있었다"만 남기고 정확한 위치는 버리므로, 계산량이 줄고 **조금 이동한 그림에도 덜 흔들립니다**.

예시 적용 4×4 특성 맵을 2×2 맥스 풀링하면 [[6, 2], [2, 9]]의 2×2가 됩니다(위 그림 오른쪽). 08-02의 CNN에서는 (32, 28, 28)이 풀링을 지나 (32, 14, 14)가 됩니다.

비유

돋보기로 숨은그림찾기

숨은그림찾기를 할 때 그림 전체를 한 번에 보지 않고, 작은 돋보기를 칸칸이 옮기며 "여기 세로선이 있나?"를 확인합니다. 찾을 모양마다 돋보기가 하나씩 있고(커널 여러 개), 다 훑은 뒤에는 "왼쪽 위 구역에 세로선 있음" 정도로만 메모합니다.

돋보기 하나	= 커널 (모든 위치에서 같은 돋보기 = 가중치 공유)
돋보기에 보이는 작은 영역	= 주변 픽셀 (지역 연결)
찾은 위치를 표시한 지도	= 특성 맵
구역별 메모	= 풀링

실습 1 합성곱과 풀링을 직접 계산하기

```
import torch
import torch.nn.functional as F

# 5×5 흑백 그림 하나 (1 = 검은 점). 가운데에 세로줄이 있다
img = torch.tensor([[0, 0, 1, 0, 0],
                    [0, 0, 1, 0, 0],
                    [0, 0, 1, 0, 0],
                    [0, 0, 1, 0, 0],
                    [0, 0, 1, 0, 0]], dtype=torch.float32)
# 3×3 커널(필터): 세로줄을 찾는 돋보기
kernel = torch.tensor([[-1, 2, -1],
                       [-1, 2, -1],
                       [-1, 2, -1]], dtype=torch.float32)

x = img.view(1, 1, 5, 5)                # (배치, 채널, 높이, 너비)
k = kernel.view(1, 1, 3, 3)
out = F.conv2d(x, k)                    # 커널을 한 칸씩 밀며 곱하고 더하기
print("합성곱 결과 (3×3):\n", out[0, 0])
print("패딩 1을 주면 크기 유지:", tuple(F.conv2d(x, k, padding=1).shape))

# 특성 맵 크기 공식: (입력 - 커널 + 2×패딩) / 스트라이드 + 1
for size, kernel_size, pad, stride in [(5, 3, 0, 1), (28, 3, 1, 1), (28, 3,
0, 2)]:
    print(f"입력 {size}, 커널 {kernel_size}, 패딩 {pad}, 스트라이드 {stride} →
{(size - kernel_size + 2 * pad) // stride + 1}")

# 맥스 풀링: 2×2 칸마다 가장 큰 값만 남겨 크기를 절반으로
fm = torch.tensor([[1., 3., 2., 0.],
                   [4., 6., 1., 1.],
                   [0., 2., 9., 7.],
                   [1., 1., 3., 8.]]).view(1, 1, 4, 4)
print("맥스 풀링 결과:\n", F.max_pool2d(fm, 2)[0, 0])
```

실행 결과

```
합성곱 결과 (3×3):
 tensor([[-3.,  6., -3.],
        [-3.,  6., -3.],
        [-3.,  6., -3.]])
패딩 1을 주면 크기 유지: (1, 1, 5, 5)
입력 5, 커널 3, 패딩 0, 스트라이드 1 → 3
입력 28, 커널 3, 패딩 1, 스트라이드 1 → 28
입력 28, 커널 3, 패딩 0, 스트라이드 2 → 13
맥스 풀링 결과:
 tensor([[6., 2.],
        [2., 9.]])
```

직접 해 보기 커널을 가로줄용(`kernel.T`)으로 바꾸면 세로줄 그림에서 큰 값이 나올까요? 커널이 "무엇을 찾는 돋보기"인지 확인해 보세요.

실습 2 채널, 다수 채널의 합성곱, 가중치 공유

```python
import torch
import torch.nn as nn

torch.manual_seed(0)
# 1) 이미지는 (채널, 높이, 너비) 3차원 텐서. 파이토치는 맨 앞에 배치를 붙여 4차원으로 받는다
gray = torch.rand(1, 1, 28, 28)        # 흑백 손글씨: 채널 1
color = torch.rand(1, 3, 28, 28)       # 컬러 사진: 빨강·초록·파랑 채널 3
print("흑백 이미지 모양:", tuple(gray.shape), " 컬러 이미지 모양:", tuple(color.shape))

# 2) 다수 채널의 합성곱: 커널도 입력과 같은 채널 수(3)를 가진다
conv_one = nn.Conv2d(in_channels=3, out_channels=1, kernel_size=3)
print("커널 1개의 가중치 모양:", tuple(conv_one.weight.shape), " → (출력 채널, 입력 채널, 높이, 너비)")
print("커널 1개 결과:", tuple(conv_one(color).shape), " → 채널별 결과를 더해 1장")

conv_four = nn.Conv2d(3, 4, kernel_size=3)       # 커널 4개 → 특성 맵 4장
print("커널 4개 결과:", tuple(conv_four(color).shape))

# 3) 가중치 공유: 같은 일을 하는 층의 학습할 숫자 개수 비교 (28×28 흑백 → 32개의 특징)
mlp = nn.Linear(28 * 28, 32)                     # 다층 퍼셉트론: 픽셀마다 따로 연결
cnn = nn.Conv2d(1, 32, kernel_size=3)            # 합성곱: 3×3 커널 32개를 이미지 전체에 돌려씀
count = lambda m: sum(p.numel() for p in m.parameters())
print(f"다층 퍼셉트론 파라미터: {count(mlp):,}개")
print(f"합성곱층 파라미터    : {count(cnn):,}개")
```

실행 결과

```
흑백 이미지 모양: (1, 1, 28, 28)  컬러 이미지 모양: (1, 3, 28, 28)
커널 1개의 가중치 모양: (1, 3, 3, 3)  → (출력 채널, 입력 채널, 높이, 너비)
커널 1개 결과: (1, 1, 26, 26)  → 채널별 결과를 더해 1장
커널 4개 결과: (1, 4, 26, 26)
다층 퍼셉트론 파라미터: 25,120개
합성곱층 파라미터    : 320개
```

직접 해 보기 `nn.Conv2d(3, 4, kernel_size=3)` 의 가중치 모양은 무엇일까요? 정의의 (C_o, C_i, K, K)로 예측한 뒤 `conv_four.weight.shape` 로 확인해 보세요.

08-01 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 이미지를 한 줄로 펴면 이웃한 픽셀 사이의 공간적 구조 정보가 사라진다. (O / X) 이름 _____

2 CNN에서는 커널의 숫자를 사람이 직접 정해 준다. (O / X) 이름 _____

3 컬러 이미지의 채널 수는? 이름 _____

① 1 ② 3 ③ 28

4 입력 채널 3, 커널 4개, 커널 크기 3×3인 합성곱층의 가중치 모양은? 이름 _____

① (3, 4, 3, 3) ② (4, 3, 3, 3) ③ (4, 3) ④ (3, 3)

5 같은 커널을 이미지 전체에 돌려쓰는 성질을 가중치 _____ 라고 한다. 이름 _____

6 입력 28, 커널 3, 패딩 1, 스트라이드 1이면 출력 크기는 _____이다. 이름 _____

7 "이미지는 주변 픽셀과 관련이 깊다"는 성질을 합성곱 연산이 어떻게 반영하는지 설명해 보세요. 이름 _____

원문: 08-01 합성곱과 풀링 — https://wikidocs.net/62306

08-02 CNN으로 MNIST 분류하기

핵심 한 줄

CNN은 합성곱층과 풀링층을 반복해 이미지를 작지만 깊은(채널이 많은) 특성 맵으로 바꾸고, 이를 펼쳐 전결합층과 소프트맥스로 분류한다.

왜 배우나

08-01의 부품을 조립해 완성된 CNN을 만들고, 숫자 하나가 층을 지날 때마다 **데이터의 모양과 내용이 어떻게 바뀌는지** 끝까지 따라가 봅니다. "특징 추출(합성곱·풀링) → 분류(전결합층)"라는 이 흐름은 이미지 모델의 기본 틀입니다.

1) CNN의 구조

정의

전형적인 CNN은 다음 순서로 데이터를 처리합니다.

1. **합성곱층**: 커널 C_o개로 특성 맵 C_o장을 만들고 ReLU를 적용합니다. 패딩을 쓰면 높이·너비는 유지되고 채널 수가 늘어납니다.
2. **풀링층**: 높이·너비를 절반으로 줄입니다. 채널 수는 그대로입니다.
3. 1~2를 반복할수록 그림은 **작아지고**(위치 정보 요약), 채널은 **많아집니다**(특징의 종류 증가). 앞쪽 층은 선·모서리 같은 단순한 무늬를, 뒤쪽 층은 그것을 조합한 복잡한 무늬를 찾습니다.
4. **펼치기(flatten)**: 마지막 특성 맵을 1차원으로 폅니다. 이 단계에서는 이미 주변 정보가 특징으로 요약되어 있습니다.
5. **전결합층 + 소프트맥스**: 클래스별 점수를 내고 확률로 바꿉니다(05장). 학습은 크로스 엔트로피로 합니다.

예시 적용

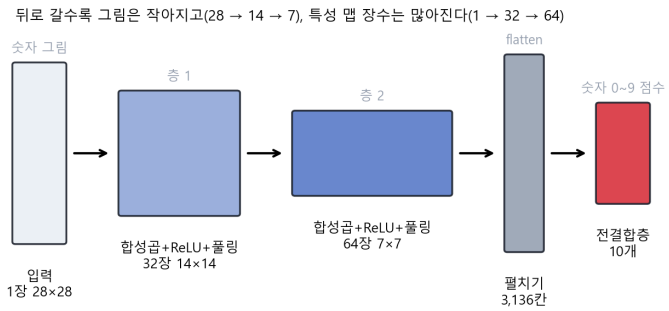


그림 7 08-02의 CNN 구조. 층을 지날 때마다 그림 크기는 절반, 특성 맵 장수는 늘어난다

2) 숫자 7 하나가 CNN을 지나는 과정

예시 적용

학습이 끝난 CNN(실습 3)에 테스트 데이터의 첫 번째 그림, 손글씨 **7**을 넣고 층마다 데이터를 꺼내 봤습니다(실습 4).

단계	하는 일	데이터 모양 (채널, 높이, 너비)	무엇이 바뀌었나
입력	흑백 손글씨 7	(1, 28, 28)	밝기 0~1의 픽셀 784개
합성곱1 + ReLU	3×3 커널 32개, 패딩 1	(32, 28, 28)	크기는 그대로, 특성 맵 32장. 각 장은 7의 가로획·세로획 등 서로 다른 무늬에 반응
맥스풀링1	2×2 구역마다 최댓값	(32, 14, 14)	높이·너비 절반. "어디쯤에 그 무늬가 있었나"만 남음
합성곱2 + ReLU	3×3×32 커널 64개	(64, 14, 14)	32장을 조합해 더 복잡한 무늬 64종을 찾음 (윗변, 사선 등)
맥스풀링2	2×2 구역마다 최댓값	(64, 7, 7)	작지만 깊은 특징 덩어리
펼치기	한 줄로	(3136,)	64 × 7 × 7 = 3,136개의 특징
전결합층	nn.Linear(3136, 10)	(10,)	숫자 0~9의 점수
소프트맥스	점수 → 확률	(10,)	7일 확률 1.00 → 예측 7

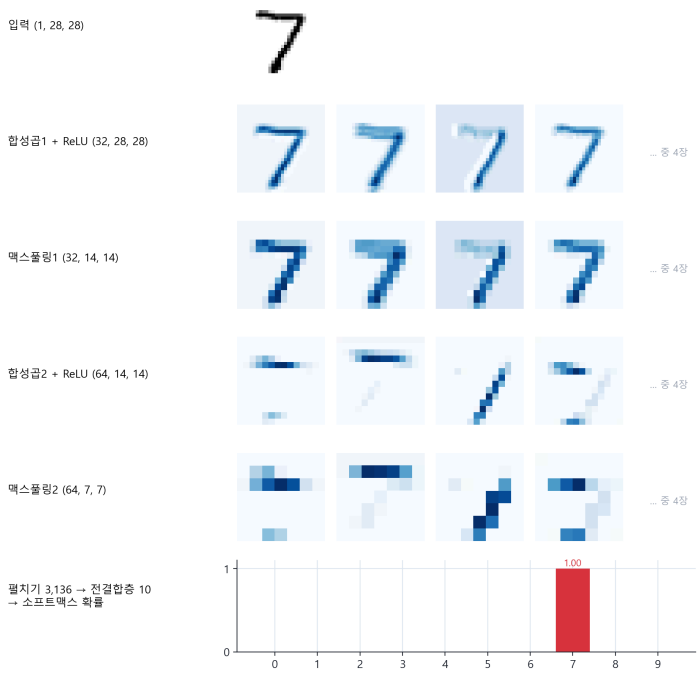


그림 8 학습된 CNN에서 꺼낸 실제 특성 맵(각 층에서 가장 강하게 반응한 4장). 뒤로 갈수록 그림은 작아지고 "윗변", "사선" 같은 부분 무늬로 쪼개지며, 마지막에 7의 확률이 1.00이 된다

첫 합성곱층의 특성 맵은 아직 7의 모양이 그대로 보이지만, 두 번째 층에서는 7의 **윗변(가로획)**, **사선(세로획)**처럼 부분 무늬로 나뉩니다. 전결합층은 "윗변이 위쪽에 있고, 사선이 오른쪽

위에서 왼쪽 아래로 내려간다"는 특징 조합을 보고 7이라고 판단합니다.

비유

공장의 조립 라인

첫 번째 작업대(합성곱층 1)는 선과 점 같은 작은 부품을 찾고, 두 번째 작업대(합성곱층 2)는 그 부품을 모아 "윗변", "사선" 같은 큰 부품을 만듭니다. 작업대 사이의 정리 담당(풀링)은 부품 목록을 간추립니다. 마지막 검수대(전결합층)는 완성된 부품 목록을 보고 "이건 7이다"라고 판정합니다.

작은 부품 찾기 = 합성곱층 1 (32장)
큰 부품 조립 = 합성곱층 2 (64장)
목록 간추리기 = 풀링
최종 판정 = 펼치기 + 전결합층 + 소프트맥스

실습 3 CNN으로 MNIST 학습하기

```
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
import torchvision.datasets as dsets
import torchvision.transforms as transforms

torch.manual_seed(777)
train = dsets.MNIST(root="MNIST_data/", train=True, transform=transforms.To
Tensor(), download=True)
test = dsets.MNIST(root="MNIST_data/", train=False, transform=transforms.To
Tensor(), download=True)
loader = DataLoader(train, batch_size=100, shuffle=True, drop_last=True)

class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        # 층 1: 합성곱(1→32장) + ReLU + 풀링  →  (32, 14, 14)
        self.layer1 = nn.Sequential(nn.Conv2d(1, 32, 3, padding=1), nn.ReLU
(), nn.MaxPool2d(2))
        # 층 2: 합성곱(32→64장) + ReLU + 풀링 →  (64, 7, 7)
        self.layer2 = nn.Sequential(nn.Conv2d(32, 64, 3, padding=1), nn.ReL
U(), nn.MaxPool2d(2))
        self.fc = nn.Linear(7 * 7 * 64, 10)        # 펼쳐서 숫자 10개 중 하나
로
    def forward(self, x):
        x = self.layer2(self.layer1(x))
        return self.fc(x.flatten(start_dim=1))     # 배치 차원만 남기고 펼치기

model = CNN()
print("입력 (1,1,28,28) → 출력", tuple(model(torch.zeros(1, 1, 28, 28)).shap
e))
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

for epoch in range(2):                             # 교재는 15번, 여기서는 2
번만
    for x, y in loader:
        loss = loss_fn(model(x), y)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    print(f"Epoch {epoch + 1}  마지막 배치 Loss: {loss.item():.4f}")

with torch.no_grad():
    x_test = test.data.float().div(255).unsqueeze(1)
    acc = (model(x_test).argmax(dim=1) == test.targets).float().mean()
print("테스트 정확도:", round(acc.item(), 4))
torch.save(model.state_dict(), "cnn_mnist.pt")      # 다음 실습(08-02b)에서
쓰려고 저장
```

실행 결과

```
입력 (1,1,28,28) → 출력 (1, 10)
Epoch 1  마지막 배치 Loss: 0.1204
Epoch 2  마지막 배치 Loss: 0.0372
테스트 정확도: 0.9841
```

직접 해 보기 06-06의 다층 퍼셉트론(3 에포크, 97.0%)과 비교해 보세요. CNN은 2 에포크만에 몇 %가 나왔나요? 코랩 CPU에서 에포크당 1~2분 걸리며, GPU로 바꾸면 훨씬 빠릅니다.

실습 4 숫자 하나를 넣고 층마다 데이터 따라가기

```
import torch
import torch.nn as nn
import torchvision.datasets as dsets
import torchvision.transforms as transforms

# 08-02에서 학습한 CNN과 같은 구조 (저장한 가중치를 불러온다)
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Sequential(nn.Conv2d(1, 32, 3, padding=1), nn.ReLU
(), nn.MaxPool2d(2))
        self.layer2 = nn.Sequential(nn.Conv2d(32, 64, 3, padding=1), nn.ReL
U(), nn.MaxPool2d(2))
        self.fc = nn.Linear(7 * 7 * 64, 10)
    def forward(self, x):
        return self.fc(self.layer2(self.layer1(x)).flatten(start_dim=1))

model = CNN()
model.load_state_dict(torch.load("cnn_mnist.pt"))
model.eval()

test = dsets.MNIST(root="MNIST_data/", train=False, transform=transforms.To
Tensor(), download=True)
x, label = test[0]                          # 테스트 첫 번째 그림
x = x.unsqueeze(0)                          # 배치 차원 추가: (1, 1, 28, 28)
print(f"정답: {label}")

# 숫자 하나가 층을 지날 때마다 모양이 어떻게 바뀌는지 따라가기
steps = [("입력", lambda t: t),
         ("합성곱1 (커널 32개)", model.layer1[0]), ("ReLU", model.layer1[1]),
("맥스풀링1", model.layer1[2]),
         ("합성곱2 (커널 64개)", model.layer2[0]), ("ReLU", model.layer2[1]),
("맥스풀링2", model.layer2[2]),
         ("펼치기", lambda t: t.flatten(start_dim=1)), ("전결합층", model.f
c)]
with torch.no_grad():
    t = x
    for name, layer in steps:
        t = layer(t)
        print(f"{name:14s} → 모양 {str(tuple(t.shape)):18s} 값 범위 {t.min
().item():6.2f} ~ {t.max().item():6.2f}")
    prob = torch.softmax(t, dim=1)[0]

top = prob.argsort(descending=True)[:3]
print("소프트맥스 확률 상위 3개:", [(d.item(), round(prob[d].item(), 4)) for d
in top])
print("예측:", top[0].item())
```

실행 결과

```
정답: 7
입력                → 모양 (1, 1, 28, 28)      값 범위   0.00 ~   1.00
합성곱1 (커널 32개)  → 모양 (1, 32, 28, 28)     값 범위  -1.37 ~   1.80
ReLU              → 모양 (1, 32, 28, 28)     값 범위   0.00 ~   1.80
맥스풀링1           → 모양 (1, 32, 14, 14)     값 범위   0.00 ~   1.80
합성곱2 (커널 64개)  → 모양 (1, 64, 14, 14)     값 범위  -7.17 ~   5.59
ReLU              → 모양 (1, 64, 14, 14)     값 범위   0.00 ~   5.59
맥스풀링2           → 모양 (1, 64, 7, 7)       값 범위   0.00 ~   5.59
펼치기              → 모양 (1, 3136)           값 범위   0.00 ~   5.59
전결합층            → 모양 (1, 10)             값 범위 -21.75 ~  14.14
소프트맥스 확률 상위 3개: [(7, 1.0), (9, 0.0), (3, 0.0)]
예측: 7
```

직접 해 보기 `test[0]`을 `test[1]`, `test[2]`로 바꿔 다른 숫자도 따라가 보세요. 모양은 매번 같고, 값과 최종 확률만 달라지나요?

확인 문제

08-02 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 풀링층을 지나면 채널 수는 그대로이고 높이·너비가 줄어든다. (O / X) 이름 _____

2 합성곱층을 지날수록 채널 수는 항상 줄어든다. (O / X) 이름 _____

3 합성곱2 + ReLU를 지난 뒤의 모양은? 이름 _____

① (32, 14, 14) ② (64, 14, 14) ③ (64, 7, 7)

4 펼치기 직전 (64, 7, 7)을 펴면 몇 개가 되나요? 이름 _____

① 448 ② 3,136 ③ 784

5 CNN의 앞쪽 층은 선·모서리 같은 _____ 무늬를, 뒤쪽 층은 그것을 조합한 복잡한 무늬를 찾는다. 이름 _____

6 실습에서 2 에포크 학습한 CNN의 테스트 정확도는 약 _____%였다. 이름 _____

7 숫자 7이 입력부터 최종 확률까지 어떤 모양 변화를 거치는지 순서대로 말해 보세요. 이름 _____

원문: 08-02 CNN으로 MNIST 분류하기 — https://wikidocs.net/63565

08-03 깊은 CNN으로 MNIST 분류하기

핵심 한 줄

합성곱층과 전결합층을 더 쌓고 드롭아웃을 넣으면 더 복잡한 무늬를 배울 수 있지만, 층을 늘린다고 항상 좋아지지는 않는다.

왜 배우나

"층을 더 쌓으면 더 똑똑해질까?"는 딥러닝에서 늘 나오는 질문입니다. 직접 깊게 쌓아 보면서, 깊이를 늘릴 때 함께 신경 써야 하는 것들 (모양 계산, 드롭아웃, train/eval 모드, 과적합)을 실제 코드로 확인합니다.

1) 깊이와 특징의 계층

정의

- CNN을 깊게 쌓으면 특징이 **계층적**으로 만들어집니다: 선·모서리 → 곡선·꼭짓점 → 부분 모양 → 전체 모양
- 층마다 풀링으로 높이·너비가 줄어드므로, 쌓을 수 있는 층 수에는 입력 크기에 따른 한계가 있습니다. 홀수 크기는 풀링 패딩으로 맞춥니다.
- 층과 가중치가 많아질수록 과적합 위험이 커지므로 **드롭아웃**(06-07)을 함께 씁니다. 드롭아웃이 있으면 학습 전 `model.train()` , 평가 전 `model.eval()` 을 꼭 호출합니다.
- 더 깊다고 항상 더 좋은 것은 아닙니다. 데이터의 복잡도에 맞는 깊이와 학습 설정 (학습률, 에포크 등)을 함께 찾아야 합니다.

예시 적용

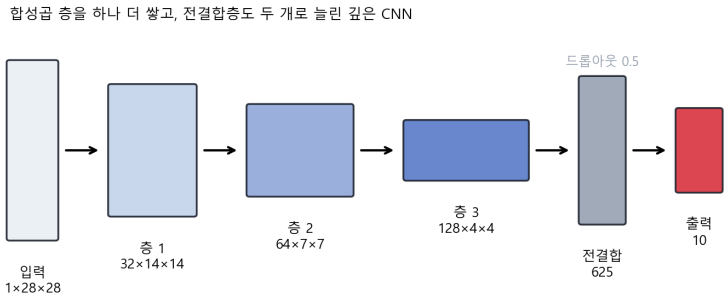


그림 9 08-03의 깊은 CNN. 합성곱층이 3개, 전결합층이 2개(사이에 드롭아웃)로 늘었다

- 합성곱층 3개: (1, 28, 28) → (32, 14, 14) → (64, 7, 7) → (128, 4, 4). 세 번째 풀링은 7을 나누기 위해 패딩 1을 줘서 4가 됩니다.
- 전결합층 2개: 2,048 → 625 → 10. 사이에 ReLU와 드롭아웃(0.5)을 넣습니다.
- 교재는 15 에포크를 학습해 98.3% 정도를 얻고, 08-02(98.8%)보다 오히려 조금 낮다고 말합니다. MNIST처럼 단순한 데이터에는 얕은 CNN으로도 충분하다는 뜻입니다. 우리 실습은 1 에포크만 학습해서 숫자가 교재와 다릅니다.

비유

돋보기를 더 많이 겹치기

현미경 렌즈를 여러 장 겹치면 더 작은 것까지 보이지만, 동네 지도를 볼 때는 오히려 불편합니다. 보고 싶은 대상에 맞는 배율이 있듯, 데이터에 맞는 깊이가 있습니다.

렌즈 장수	= 층의 깊이
대상에 맞는 배율	= 데이터에 맞는 모델 크기
초점 맞추기	= 학습률, 드롭아웃, 에포크 조절

실습 5 깊은 CNN의 모양 추적과 학습

```
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
import torchvision.datasets as dsets
import torchvision.transforms as transforms

torch.manual_seed(777)
train = dsets.MNIST(root="MNIST_data/", train=True, transform=transforms.To
Tensor(), download=True)
test = dsets.MNIST(root="MNIST_data/", train=False, transform=transforms.To
Tensor(), download=True)
loader = DataLoader(train, batch_size=100, shuffle=True, drop_last=True)

def conv_block(c_in, c_out, pad_pool=0):
    return nn.Sequential(nn.Conv2d(c_in, c_out, 3, padding=1), nn.ReLU(),
                         nn.MaxPool2d(2, padding=pad_pool))

class DeepCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.convs = nn.Sequential(conv_block(1, 32),           # 28 → 14
                                   conv_block(32, 64),          # 14 → 7
                                   conv_block(64, 128, 1))      # 7 → 4
        self.fcs = nn.Sequential(nn.Flatten(),
                                 nn.Linear(4 * 4 * 128, 625), nn.ReLU(), n
n.Dropout(0.5),
                                 nn.Linear(625, 10))
    def forward(self, x):
        return self.fcs(self.convs(x))

model = DeepCNN()
x = torch.zeros(1, 1, 28, 28)
for i, block in enumerate(model.convs, 1):                    # 층마다 모양
추적
    x = block(x)
    print(f"합성곱 층 {i} 뒤:", tuple(x.shape))

loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
model.train()
for epoch in range(1):                                        # 교재는 15번,
여기서는 1번만
    for x, y in loader:
        loss = loss_fn(model(x), y)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    print(f"Epoch {epoch + 1}  마지막 배치 Loss: {loss.item():.4f}")

model.eval()                                                  # 드롭아웃 끄기
with torch.no_grad():
    x_test = test.data.float().div(255).unsqueeze(1)
    acc = (model(x_test).argmax(dim=1) == test.targets).float().mean()
print("테스트 정확도:", round(acc.item(), 4))
```

실행 결과

```
합성곱 층 1 뒤: (1, 32, 14, 14)
합성곱 층 2 뒤: (1, 64, 7, 7)
합성곱 층 3 뒤: (1, 128, 4, 4)
Epoch 1  마지막 배치 Loss: 0.0279
테스트 정확도: 0.9868
```

직접 해 보기 `conv_block(64, 128, 1)`의 마지막 1(풀링 패딩)을 0으로 바꾸면 층 3 뒤의 모양은 어떻게 되고, 어떤 에러가 날까요?

확인 문제

08-03 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 층을 많이 쌓을수록 정확도는 항상 올라간다. (O / X) 이름 _____

2 드롭아웃을 쓰는 모델은 평가 전에 model.eval()을 호출해야 한다. (O / X) 이름 _____

3 깊은 CNN의 합성곱층 개수는? 이름 _____

① 1개 ② 2개 ③ 3개 ④ 5개

4 CNN을 깊게 쌓을 때 특징이 만들어지는 방식으로 알맞은 것은? 이름 _____

① 선·모서리 → 부분 모양 → 전체 모양 ② 전체 모양 → 선 ③ 층마다 같은 특징

5 실습에서 합성곱 층 3을 지난 뒤의 모양은 (1, 128, _____, _____)이다. 이름 _____

6 교재에 따르면 15 에포크 학습한 깊은 CNN은 08-02의 CNN보다 정확도가 조금 _____. 이름 _____

7 렌즈 비유로 "데이터에 맞는 깊이"를 설명해 보세요. 이름 _____

원문: 08-03 깊은 CNN으로 MNIST 분류하기 — https://wikidocs.net/63618

오늘의 요약

다시 보기



정의 모음

개념	정의	MNIST 예시
공간적 구조	가까운 픽셀끼리 연관이 깊은 이미지의 성질. 펴면 사라진다	위 이웃이 펴면 28칸 앞으로
채널	한 픽셀을 표현하는 값의 개수. 이미지 = (높이, 너비, 채널)	흑백 1채널, 컬러 3채널
합성곱	커널을 밀며 주변 픽셀과 곱해 더하는 연산. 가중치 공유	파라미터 25,120 → 320
출력 크기	$\lfloor (I - K + 2P)/S \rfloor + 1$	28, 커널 3, 패딩 1 → 28
다수 채널	커널 채널 = 입력 채널, 출력 채널 = 커널 개수	Conv2d(32, 64): 32장 → 64장
풀링	구역마다 값 하나만 남겨 크기 축소. 가중치 없음	(32, 28, 28) → (32, 14, 14)
CNN 흐름	합성곱·풀링 반복 → 펼치기 → 전결합 → 소프트맥스	(1,28,28) → ⋯ → (64,7,7) → 3136 → 10

다음

다음 자료 예고: 09장 자연어 처리 기초 (10/8)

드디어 문장을 다룹니다. 그림은 처음부터 숫자였지만 문장은 숫자가 아닙니다. 09장에서는 문장을 토큰으로 자르고, 다듬고, 번호를 붙여 신경망에 넣을 수 있는 숫자로 바꾸는 방법을 배웁니다.

정답과 해설

08-01 합성곱과 풀링

1. O — 바로 위아래 이웃 픽셀이 28칸씩 떨어진 칸이 됩니다.
2. X — 커널의 숫자는 학습할 가중치이고, 학습으로 정해집니다.
3. ② — 빨강, 초록, 파랑
4. ② — $(C_o, C_i, K, K) = (4, 3, 3, 3)$
5. 공유
6. 28 — $(28 - 3 + 2) \div 1 + 1 = 28$
7. 예시 답: 출력의 한 칸을 계산할 때 커널 크기만큼의 주변 픽셀을 함께 곱해 더하므로, 이웃한 픽셀들이 만드는 선·모서리 같은 무늬가 그대로 반영된다.

08-02 CNN으로 MNIST 분류하기

1. O
2. X — 이 CNN에서는 1 → 32 → 64로 늘어납니다.
3. ②
4. ② — 64 × 7 × 7 = 3,136
5. 단순한(저수준)
6. 98.4 (실행 결과 0.9841)
7. 예시 답: (1, 28, 28) → 합성곱1 (32, 28, 28) → 풀링 (32, 14, 14) → 합성곱2 (64, 14, 14) → 풀링 (64, 7, 7) → 펼치기 3,136 → 전결합 10 → 소프트맥스로 7일 확률 1.00

08-03 깊은 CNN으로 MNIST 분류하기

1. X
2. O — 평가할 때 드롭아웃이 꺼져야 모든 뉴런을 씁니다.
3. ③
4. ①
5. 4, 4
6. 낮았다
7. 예시 답: 렌즈를 많이 겹치면 더 작은 것을 보지만 큰 대상을 볼 땐 불편하듯, 모델도 데이터의 복잡도에 맞는 깊이가 있다.

용어 정리

용어	정의
공간적 구조	가까운 픽셀끼리 값이 밀접하게 연관된 이미지의 성질
채널 (깊이)	한 픽셀을 표현하는 값의 개수. 흑백 1, 컬러 3, 합성곱 뒤에는 특징의 종류
합성곱 (convolution)	커널을 이미지 위로 밀며 겹치는 부분을 곱해 더하는 연산
커널 (필터)	무늬를 찾는 작은 가중치 행렬. 학습으로 정해진다
특성 맵	합성곱 결과로 나온 표. 무늬가 어디에 있는지 보여 준다
가중치 공유	같은 커널을 이미지 전체에 돌려쓰는 것. 파라미터가 적어진다
패딩 / 스트라이드	테두리에 0 두르기 / 커널을 한 번에 미는 칸 수
맥스 풀링	구역마다 가장 큰 값만 남겨 크기를 줄이는 연산
전결합층	모든 입력이 모든 출력과 연결된 층 (nn.Linear)
(배치, 채널, 높이, 너비)	파이토치 이미지 텐서의 모양 순서